



## تغییر جدول

بخش چهارم: مقدمات پیاده‌سازی و SQL

### □ دستور تغییر جدول ALTER TABLE

نام جدول

**ALTER TABLE** *tableName*

اضافه کردن ستون، تغییر تعریف ستون، حذف ستون و ...

**[ADD [COLUMN] *columnName* *dataType* [NOT NULL] [UNIQUE]**

اضافه کردن ستون جدید

**[DEFAULT *defaultOption*] [CHECK (*searchCondition*)] ]**

**[DROP [COLUMN] *columnName* [RESTRICT | CASCADE] ]**

حذف کردن ستون

**[ADD [CONSTRAINT [*constraintName*]] *tableConstraintDefinition***

اضافه کردن قید

**[DROP [CONSTRAINT *constraintName* [RESTRICT | CASCADE] ]**

حذف کردن قید

**[ALTER [COLUMN] SET DEFAULT *defaultOption*]**

اضافه کردن مقدار پیش فرض

**[ALTER [COLUMN] DROP DEFAULT]**

حذف کردن مقدار پیش فرض

...

اضافه کردن ستون «وضعیت» به جدول اطلاعات دانشجو



**ALTER TABLE STT**

**ADD COLUMN STATE CHAR(10)**

## مثال: یک نمونه از کد Alter Table

**جدول** Customers (CustomerID, CustomerName, City, Telephone, Email)

|                                                        |                                               |
|--------------------------------------------------------|-----------------------------------------------|
| ALTER TABLE Customers                                  | تغییر جدول Customer                           |
| DROP COLUMN Telephone;                                 | حذف ستون با نام Telephone                     |
| ADD COLUMN DateOfBirth date;                           | اضافه کردن ستون با نام DateOfBirth و نوع date |
| ALTER COLUMN CustomerName Cname;                       | تغییر نام ستون CustomerName به Cname          |
| ALTER COLUMN City SET DEFAULT 'Londan';                | تعریف مقدار پیش فرض برای ستون City            |
| ALTER COLUMN <u>City</u> DROP DEFAULT;                 | حذف مقدار پیش فرض برای ستون City              |
| ADD CONSTRAINT email_unique UNIQUE (Email);            | تعیین محدودیت UNIQUE برای ستون Email          |
| ADD CONSTRAINT constraint_PR PRIMARY KEY (CustomerID); | تعیین ستون CustomerID به عنوان کلید اصلی      |
| DROP CONSTRAINT email_unique;                          | حذف محدودیت UNIQUE برای ستون Email            |

**نکته:** در پایگاه داده‌های مختلف ممکن است syntax کد تا حدودی متفاوت باشد.



## زبان جدولی TDBL

بخش چهارم: مقدمات پیاده‌سازی و SQL

□ عملیات در TDB : دستورهای DML

SELECT

بازیابی

INSERT

درج

DELETE

حذف

UPDATE

به‌نگام‌سازی

Select نام ستونها

From نام جداول

Where شرط

کوئری ۲: شماره دانشجویی و نام دانشجویان از جدول دانشجو که کارشناسی رشته فیزیک هستند را در خروجی نشان بده. ستون شماره دانشجویی و نام دانشجو در خروجی به ترتیب SN و SName باشد.

نام دانشجو شماره دانشجویی

**مثال**

```
SELECT STID , STNAME
FROM STT
WHERE STMJR='phys'
AND
STLEV='bs'
```

جدول دانشجو رشته فیزیک مقطع کارشناسی

خروجی

| STID    | STNAME  |
|---------|---------|
| 9800666 | Ahmadi  |
| 9800667 | Nori    |
| 9800665 | Soltani |

**مثال**

```
SELECT STT.STID AS SN,
STT.STNAME AS SName
FROM STT
WHERE STT.STMJR='phys'
AND
STT.STLEV='bs'
```

خروجی

| SN      | SName   |
|---------|---------|
| 9800666 | Ahmadi  |
| 9800667 | Nori    |
| 9800665 | Soltani |

نکته: از نام جدول به همراه یک نقطه قبل از نام ستون می توان استفاده کرد (اختیاری)  
نکته: با دستور AS می توان نام ستون در خروجی را تغییر داد.

## بازیابی داده‌ها (ادامه)



بخش چهارم: مقدمات پیاده‌سازی و SQL

تمام اطلاعات تهیه کنندگان را در خروجی نشان بده



**SELECT \*** تمام ستونها  
و براساس نام تهیه کننده مرتب باشد.  
**FROM S** جدول تهیه کننده  
مرتب شده: ■

**ORDER BY SNAME** یا 2  
↓  
شماره ستون  
پیش فرض صعودی: (Ascending) ■  
نزولی (Descending): باید قید شود. ■

نکته: ستاره \* بعد از دستور Select به معنای انتخاب تمام ستون‌ها است.

S (S#, Sname, City, Status) جدول تهیه کننده

شهر شماره تهیه کننده

قابلیت‌های پیشرفته (Advanced features):



**SELECT S#, CITY**

**FROM S** جدول تهیه کننده  
**WHERE SNAME**  $\left\{ \begin{array}{l} \text{LIKE} \\ \text{NOT LIKE} \end{array} \right\} \left\{ \begin{array}{l} \text{'\%N'} \rightarrow \text{با N تمام شود} \\ \text{'M\%'} \rightarrow \text{با M شروع شود} \\ \text{'\_A\_'} \rightarrow \text{دقیقا 5 کاراکتر، کاراکتر سوم A} \end{array} \right.$

مثال: **Where SNAME LIKE '%N'** بجای حروفی که مشخص نشده، underline می‌گذاریم نه space. 15 نکته: Activ

Go to



```
SELECT P#      شماره قطعه  
FROM P        جدول قطعه  
WHERE WEIGHT BETWEEN (5,15)  
یا  
WHERE WEIGHT >= 5 AND WEIGHT <= 15
```

**BETWEEN**



□ شماره قطعاتی را بدهید که وزن آنها بین ۵ و ۱۵ است.

جدول قطعه P (P#, PName, Color, Weight)



## بازیابی داده‌ها (ادامه)

بخش چهارم: مقدمات پیاده‌سازی و SQL

NULL



شهر شماره تهیه کننده

جدول تهیه کننده  
**SELECT S#, CITY**

**FROM S**

**WHERE STATUS**

**IS NULL**

**IS NOT NULL**

مثال: شماره تهیه کنندگان و شهر آنها، به شرط آن که مقدار صفت STATUS برابر NULL باشد.

شهر شماره تهیه کننده

**SELECT S#, CITY**

**FROM S** جدول تهیه کننده

**WHERE STATUS IS NULL**

S (S#, Sname, City, Status) جدول تهیه کننده

```
SELECT * FROM CustomersTbl  
Where CustomerID > 2
```

مثال:

تمام اطلاعات مشتریانی که شناسه آنها بزرگتر از ۲ است از جدول CustomersTbl نشان بده.

```
SELECT * FROM CustomersTbl  
Where Cname = N'حسن' AND CFamily = N'اکبرزاده'
```

```
SELECT * FROM CustomersTbl  
Where CName in (N'حسن', N'مهسا')
```

تمام اطلاعات مشتریانی که نام آنها حسن و نام خانوادگی آنها اکبرزاده است از جدول CustomersTbl

```
SELECT * FROM CustomersTbl  
Where CustomerID = 2 OR  
CustomerID = 4
```

تمام اطلاعات مشتریانی که شناسه آنها ۲ یا ۴ است.

تمام اطلاعات مشتریانی از جدول CustomersTbl که بر اساس نام خانوادگی بصورت صعودی مرتب باشد.

```
SELECT * FROM CustomersTbl  
order by CFamily Asc
```

```
SELECT * FROM CustomersTbl  
order by Cfamily, CustomerID Desc
```

تمام اطلاعات مشتریانی از جدول CustomersTbl که بر اساس

نام خانوادگی بصورت صعودی و بر اساس شناسه مشتری بصورت نزولی

مرتب باشد. اول همه ردیف‌ها بر اساس نام خانوادگی بصورت صعودی مرتب می‌شود و اگر چند نام خانوادگی یکسان داشتیم آنها را براساس

```
SELECT CustomerID, CName, CFamily, CMobile, CTel,  
CCity FROM CustomersTbl  
order by 3, 1 Desc
```

شناسه مشتری بصورت نزولی مرتب می‌کند.

اطلاعات ستونهای CustomerID, CName, CFamily, CMobile, CTel, و CCity

از جدول CustomerTbl که بر اساس ستون سوم صعودی و براساس ستون اول نزولی مرتب باشد



```
SELECT column1, column2, ...
```

```
FROM table1
```

**EXCEPT or UNION or INTERSECT**

```
SELECT column1, column2, ...
```

```
FROM table2;
```

## اجتماع، اشتراک و تفاضل

- نکته:** اگر بعد از UNION و دو دستور دیگر، کلمه ALL نوشته شود. تمام نتایج در خروجی آورده می‌شود. در غیر این صورت نتایج تکراری حذف می‌شود.
- ستون‌های همه Query ها باید تعداد و ترتیب یکسانی داشته باشند.
  - نوع داده ستون‌های متناظر باید باهم یکسان یا سازگار باشند.

**مثال:**

سوال: حاصل اجتماع، اشتراک و تفاضل ستون S# از جدول S و SP چه می‌شود؟

جدول تهیه‌کننده S

| S#  | SNAME | ... |
|-----|-------|-----|
| s1  | sn1   | ... |
| s2  | sn2   | ... |
| s3  | sn3   | ... |
| s4  | sn4   | ... |
| ... | ...   | ... |

جدول تهیه‌کننده-قطعه SP

| S# | P# | QTY |
|----|----|-----|
| s1 | p1 | 100 |
| s1 | p2 | 120 |
| s1 | p3 | 500 |
| s2 | p1 | 50  |
|    |    |     |



### Aggregation Functions ☐

AVG ☐ ← میانگین

MIN ☐ ← مینیمم

MAX ☐ ← ماکزیمم

SUM ☐ ← جمع

COUNT(\*) / COUNT(Column) ☐ ← تعداد عبارات ناهیچمقدار / تعداد کل سطرها

بیشینه وضعیت تهیه کنندگان در شهرهای c1 یا c2



```
SELECT MAX ( STATUS ) AS SMAX  
FROM S  
WHERE CITY='c1'  
OR  
CITY='c2'
```

جدول تهیه کننده S (S#, Sname, City, Status)

نکته: خروجی یک عدد است که بیشترین مقدار را نشان می دهد

و در یک ستون بنام SMAX نشان داده می شود.



تعداد انواع قطعات تولیدی توسط تولیدکنندگان



```
SELECT COUNT (DISTINCT P#)
FROM SP
```

تعداد انواع قطعات قابل تولید



```
SELECT COUNT (*)
FROM P
```

تعداد کل قطعات تولیدی توسط s2



```
SELECT SUM (QTY)
FROM SP
WHERE S# = 's2'
```

SP

P

| S# | P# | QTY | P# | Pname | Color  |
|----|----|-----|----|-------|--------|
| s1 | p1 | 100 | P1 | Pn1   | Red    |
| s1 | p2 | 120 | P2 | Pn2   | Green  |
| s1 | p3 | 500 | P3 | Pn3   | Blue   |
| s2 | p1 | 50  | P4 | Pn4   | Black  |
|    |    |     | P5 | Pn5   | Yellow |

S (S#, Sname, City, Status) جدول تهیه کننده

P (P#, Pname, Color) جدول قطعه

SP (S#, P#, QTY) جدول تهیه کننده-قطعه



## GROUP BY

سطرهای جدول داده شده در کلاز FROM را گروه بندی می کند، به نحوی که مقدار ستون(های) گروه بندی در گروه یکسان است.

تعداد کل قطعات تولیدی توسط هر تولیدکننده



```
SELECT S# , SUM(QTY) AS SQ
FROM SP
GROUP BY S#
```

جدول جواب

| S#  | SQ  |
|-----|-----|
| s1  | 280 |
| s2  | 100 |
| s3  | 203 |
| ... | ... |

SP  
گروه بندی  
شده

| S#  | P#  | QTY |
|-----|-----|-----|
| s1  | p1  | 100 |
| s1  | p2  | 100 |
| s1  | p4  | 80  |
| s2  | p2  | 50  |
| s2  | p3  | 50  |
| s3  | p5  | 203 |
| ... | ... | ... |





## گروه‌بندی (ادامه)

بخش چهارم: مقدمات پیاده‌سازی و SQL

### HAVING

 امکانی است برای دادن شرط یا شرایط ناظر به گروه سطرها

شماره تهیه‌کنندگانی را بدهید که بیش از ۱۰۰ قطعه تولید کرده‌اند.



SP

| S# | P# | QTY |
|----|----|-----|
| s1 | p1 | 100 |
| s1 | p2 | 120 |
| s1 | p3 | 500 |
| s2 | p1 | 50  |
| s2 | p2 | 20  |

```
SELECT S#  
  
FROM SP  
  
GROUP BY S#  
  
HAVING SUM(QTY) > 100
```

**نکته:** Where شرط روی سطر است و having شرط داخل هر گروه است.

**نکته:** جلوی دستور select فقط می‌توان از ستون‌های معرفی شده در دستور Group و یا ستون‌های شامل توابع تجمعی استفاده کرد. مثال زیر غلط است.

```
SELECT S# , P#  
  
FROM SP  
  
GROUP BY S#
```



## بازیابی از بیش از یک جدول – عملگر پیوند یا JOIN

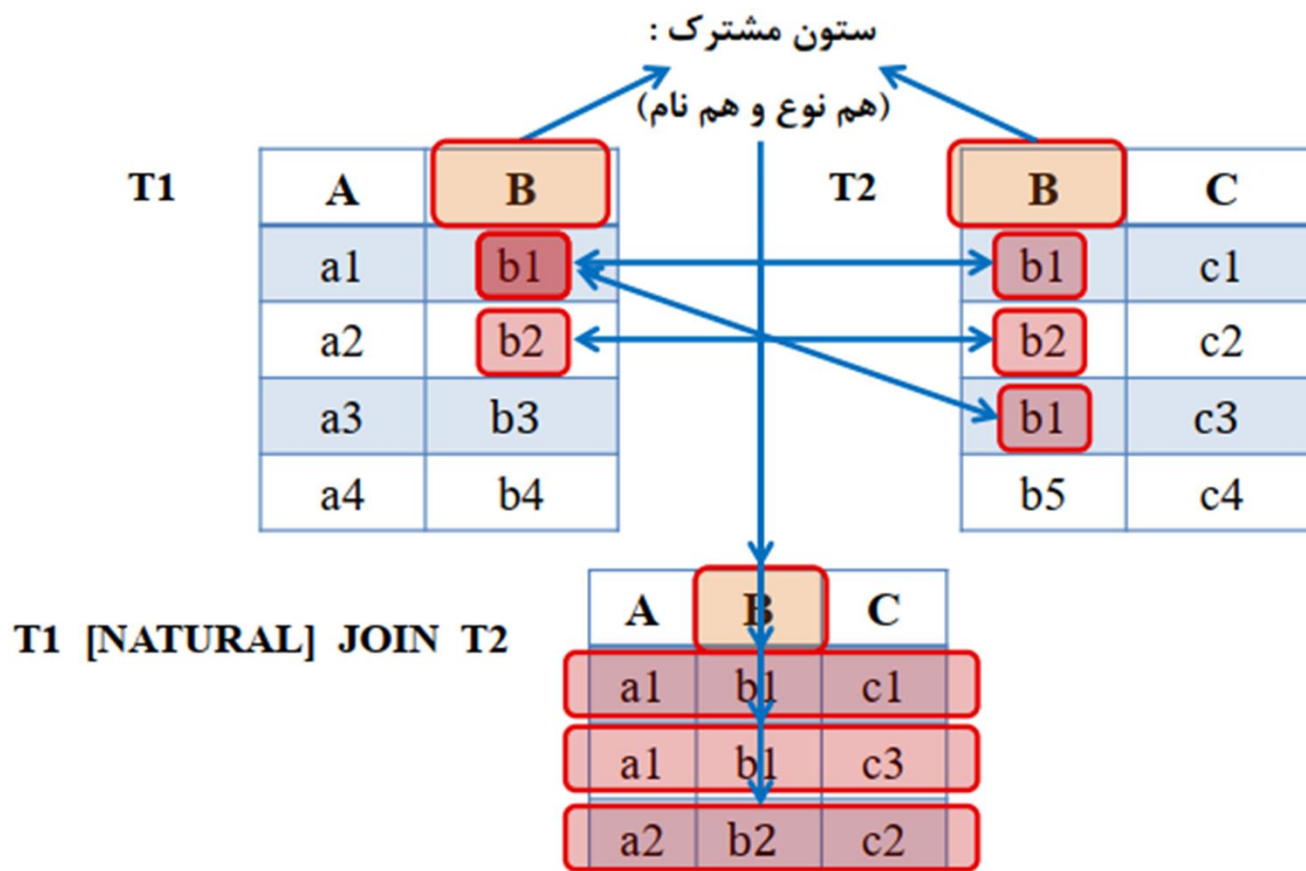
بخش چهارم: مقدمات پیاده‌سازی و SQL

**نکته:** فقط مقادیر مشابه ترکیب می‌شوند.

**پیوند:** ارائه مقدماتی

**نکته:** ستون مشترک یک بار در جدول نتیجه آورده می‌شود.

T1 [NATURAL] JOIN T2





## بازیابی از بیش از یک جدول – عملگر پیوند یا JOIN (ادامه)

بخش چهارم: مقدمات پیاده‌سازی و SQL

توضیح مقدماتی عملگر پیوند:

صرف نظر از جزئیات تئوریک، سطرهای دو جدول را که مقدار ستون(های) مشترکشان یکسان است، به هم پیوند می‌زند.

```
SELECT SNAME
FROM S [NATURAL] JOIN SP
WHERE P# = 'p2'
```

نام تهیه کنندگان قطعه 'p2' را بدهید:



| S   |       |     |
|-----|-------|-----|
| S#  | SNAME | ... |
| s1  | sn1   | ... |
| s2  | sn2   | ... |
| s3  | sn3   | ... |
| s4  | sn4   | ... |
| ... | ...   | ... |

| SP |    |     |
|----|----|-----|
| S# | P# | QTY |
| s1 | p1 | 100 |
| s1 | p2 | 120 |
| s1 | p3 | 500 |
| s2 | p1 | 50  |
|    |    |     |

| S [NATURAL] JOIN SP |       |     |    |     |
|---------------------|-------|-----|----|-----|
| S#                  | SNAME | ... | P# | QTY |
| s1                  | sn1   | ... | p1 | 100 |
| s1                  | sn1   | ... | p2 | 120 |
| s1                  | sn1   | ... | p3 | 500 |
| s2                  | sn2   | ... | p1 | 50  |
| ...                 | ...   | ... |    |     |



## عملیات ذخیره سازی

بخش چهارم: مقدمات پیاده سازی و SQL

### دستورهای INSERT, UPDATE, DELETE ☐

#### درج INSERT: ☐

نام ستون ها      نام جدول  
**INSERT INTO** *table-name* [(*col1,col2, ...*)]  
**VALUES** (*one row*) | *subquery*  
مقادیر ستون ها

#### بهنگام سازی UPDATE: ☐

نام جدول  
**UPDATE** *table-name*  
مقدار      نام ستون  
**SET** *col = value / expression* [, *col = value / expression*] ...  
:  
شرط  
**WHERE** *condition(s) / subquery*

#### حذف DELETE: ☐

نام جدول  
**DELETE FROM** *table-name*  
شرط  
**WHERE** *condition(s) / subquery*



درج سطری (سطر کامل – سطر ناقص): ستون های جدول دانشجو جدول دانشجو

```
INSERT INTO STT (STID, STNAME, STMJR, STLEV, STDEID)
VALUES ( '222' , 'st2' , 'IT' , 'bs' , 'D17' )
```

مقادیر یک رکورد اطلاعاتی

```
INSERT INTO STT
VALUES ( '222' , 'st2' , 'IT' , 'bs' , 'D17' )
```

**نکته:** نوشتن نام ستون های جدول اختیاری است.  
مقادیر به ترتیب در ستون ها قرار می گیرد.

```
INSERT INTO STT
VALUES ( '333' , 'st3' , Null , 'ms' , Null )
```

```
INSERT INTO STT
VALUES ( '333' , 'st3' )
```

**نکته:** اگر تعداد مقادیر از ستون ها کمتر باشد.  
مقادیر به ترتیب در ستون ها قرار می گیرد.

```
INSERT INTO STT
VALUES ( '222' , 'st2' , 'IT' , 'bs' , 'D17' )
( '223' , 'st4' , 'AI' , 'ms' , 'D17' )
```

**نکته:** می توان همزمان چند رکورد اطلاعاتی را اضافه کرد.



## بهنگام سازی

بخش چهارم: مقدمات پیاده سازی و SQL

بهنگام سازی چند سطر:



تعداد واحد تمام درس های عملی گروه آموزشی D11 را برابر یک کن.

```
UPDATE COT
SET CREDIT = '1'
WHERE COTYPE = 'p' AND CODEID = 'D11'
```

دپارتمان درس    نوع درس    تعداد واحد درس    عنوان درس    شماره درس

جدول درس COT (COID, COTITLE, CREDIT, COTYPE, CODEID)



حذف

بخش چهارم: مقدمات پیاده‌سازی و SQL

از جدول انتخاب درس com111 را برای دانشجوی 88104444 حذف کنید.



**DELETE FROM SCT**

**WHERE STID = 88104444**

**AND**

**COID = 'COM111'**

**SCT (STID, COID, TR, YR, GRADE)** جدول انتخاب



تمرین : شماره دانشجویانی را بدهید که در ترم دوم سال ۸۷-۸۸ بیش از ۲۰ واحد گرفته باشند.

تمرین : شماره دانشجویانی را بدهید که در ترم دوم سال ۸۷-۸۸ بیش از ۷ درس گرفته باشند.

```
SELECT STID
FROM SCT NATURAL JOIN COT
WHERE TR='2' AND YR='88-87'
GROUP BY STID
HAVING SUM(CREDIT)>20
```

جدول دانشجو STT (STID, STNAME, STMJR, STLEV, STDEID)

جدول درس COT (COID, COTITLE, CREDIT, COTYPE, CODEID)

جدول انتخاب SCT (STID, COID, TR, YR, GRADE)

```
SELECT STID
FROM SCT
WHERE TR='2' AND YR='88-87'
GROUP BY STID
HAVING COUNT (*)>7
```

```
INSERT INTO BrandsTbl  
(BrandName) VALUES (N'گلرنگ')
```

داخل جدول BrandsTbl، و ستون brandName مقدار گلرنگ را اضافه میکند.

```
Update CustomersTbl SET  
CTel= '75230052', Cmobile = '09115891005'  
Where CustomerID = 1
```

داخل جدول CustomerTbl، و برای مشتری که CustomerID  
آن مقدار یک دارد، مقدار ستون CTel با '75230052' و مقدار ستون  
Cmobile با '09115891005' آپدیت می‌شود.

```
Update CustomersTbl SET  
CCity= N'تهران'
```

**نکته ۱:** برای مقداردهی رشته‌های متنی از انواع داده‌ای مانند Char، Nchar، Nvarchar و ... بایستی آن‌ها را در بین دو کاراکتر '' قرار دهیم.

داخل جدول CustomerTbl، مقدار ستون CCity با 'تهران'  
آپدیت می‌شود. چون شرط where نداریم تمام مقادیر ستون CCity  
با این مقدار آپدیت می‌شود.

```
DELETE From BrandsTbl  
Where BrandName = N'اوه'
```

**نکته ۲:** کاربرد حرف N قبل از رشته‌های متنی و چسبیده به کاراکتر  
(Single Quotation) بعد آن برای این است که در زبان SQL برای درج و  
نمایش این که یک رشته متنی Unicode است مانند اسامی فارسی بایستی قبل  
از این رشته‌های متنی از حرف N استفاده کنید. در غیر این صورت ممکن  
است در جستجو یا نمایش این مقادیر به مشکل برخورد کنید. توجه داشته  
باشید که این حرف حتماً باید به شکل حرف بزرگ و چسبیده به کاراکتر  
باشد و پس از نوشتن در محیط نرم‌افزار به رنگ قرمز نشان داده شود.

از جدول BrandsTbl، و سطرهایی که مقدار  
ستون brandName برابر گلرنگ است حذف می‌شود.



پایان